UART TRANSMITTER

```
module uart_tx #(
    parameter CLKS_PER_BIT = 104
)(
    input wire clk,
    input wire reset,
    input wire tx_start,
    input wire [7:0] tx_data,

    output reg tx_serial,
    output reg tx_busy,
    output reg tx_done
);

    localparam IDLE  = 2'd0;
    localparam START = 2'd1;
    localparam DATA  = 2'd2;
    localparam STOP  = 2'd3;

    reg [1:0] state;
    reg [15:0] clk_count;
    reg [2:0] bit_index;
    reg [7:0] data_reg;

    always @(posedge clk) begin
        if (reset) begin
            state     <= IDLE;
            clk_count  <= 0;
            bit_index  <= 0;
```

```
    data_reg   <= 0;
    tx_serial  <= 1'b1;
    tx_busy    <= 1'b0;
    tx_done    <= 1'b0;
end else begin
    tx_done <= 1'b0;

    case (state)

        IDLE: begin
            tx_serial <= 1'b1;
            tx_busy   <= 1'b0;
            clk_count <= 0;
            bit_index <= 0;

            if (tx_start) begin
                data_reg  <= tx_data;
                tx_busy   <= 1'b1;
                tx_serial <= 1'b0;
                state     <= START;
            end
        end

        START: begin
            if (clk_count == CLKS_PER_BIT - 1) begin
                clk_count <= 0;
                bit_index <= 0;
                tx_serial <= data_reg[0];
                state     <= DATA;
```

```
    end else begin
      clk_count <= clk_count + 1'b1;
    end
  end

  DATA: begin
    if (clk_count == CLKS_PER_BIT - 1) begin
      clk_count <= 0;

      if (bit_index == 3'd7) begin
        tx_serial <= 1'b1;
        state     <= STOP;
      end else begin
        bit_index <= bit_index + 1'b1;
        tx_serial <= data_reg[bit_index + 1'b1];
      end
    end else begin
      clk_count <= clk_count + 1'b1;
    end
  end

  STOP: begin
    if (clk_count == CLKS_PER_BIT - 1) begin
      clk_count <= 0;
      tx_serial <= 1'b1;
      tx_busy   <= 1'b0;
      tx_done   <= 1'b1;
      state     <= IDLE;
    end else begin
```

```
                clk_count <= clk_count + 1'b1;
            end
        end

        default: begin
            state     <= IDLE;
            tx_serial <= 1'b1;
            tx_busy   <= 1'b0;
            clk_count <= 0;
        end

    endcase
  end
end

endmodule
```

UART RECEIVER

```
module uart_rx #(
    parameter CLKS_PER_BIT = 104
)(
    input wire clk,
    input wire reset,
    input wire rx_serial,

    output reg [7:0] rx_data,
    output reg rx_valid
);
```

```
localparam IDLE  = 2'd0;
localparam START = 2'd1;
localparam DATA  = 2'd2;
localparam STOP  = 2'd3;

reg [1:0] state;
reg [15:0] clk_count;
reg [2:0] bit_index;

always @(posedge clk) begin
    if (reset) begin
        state     <= IDLE;
        clk_count <= 0;
        bit_index <= 0;
        rx_data   <= 0;
        rx_valid  <= 1'b0;
    end else begin
        rx_valid <= 1'b0;

        case (state)

            IDLE: begin
                clk_count <= 0;
                bit_index <= 0;

                if (rx_serial == 1'b0) begin
                    state <= START;
                end
            end
```

```
START: begin
    if (clk_count == (CLKS_PER_BIT / 2) - 1) begin
        clk_count <= 0;

        if (rx_serial == 1'b0) begin
            state     <= DATA;
            bit_index <= 0;
        end else begin
            state <= IDLE;
        end
    end else begin
        clk_count <= clk_count + 1'b1;
    end
end

DATA: begin
    if (clk_count == CLKS_PER_BIT - 1) begin
        clk_count <= 0;
        rx_data[bit_index] <= rx_serial;

        if (bit_index == 3'd7) begin
            state <= STOP;
        end else begin
            bit_index <= bit_index + 1'b1;
        end
    end else begin
        clk_count <= clk_count + 1'b1;
    end
```

```
            end

            STOP: begin
               if (clk_count == CLKS_PER_BIT - 1) begin
                  clk_count <= 0;
                  state     <= IDLE;

                  if (rx_serial == 1'b1) begin
                     rx_valid <= 1'b1;
                  end
               end else begin
                  clk_count <= clk_count + 1'b1;
               end
            end

            default: begin
               state     <= IDLE;
               clk_count <= 0;
            end

         endcase
      end
   end

endmodule
```

TESTBENCH

```
`timescale 1ns/1ps
```

```
module tb_uart;

    parameter CLKS_PER_BIT = 104;

    reg clk;
    reg reset;
    reg tx_start;
    reg [7:0] tx_data;

    wire tx_serial;
    wire tx_busy;
    wire tx_done;

    wire [7:0] rx_data;
    wire rx_valid;

    integer errors;
    integer timeout;
    integer tests_passed;

    // Clock generation: 1 MHz
    initial begin
        clk = 1'b0;
        forever #500 clk = ~clk;
    end

    // UART transmitter
    uart_tx #(
        .CLKS_PER_BIT(CLKS_PER_BIT)
```

```
) transmitter (
    .clk(clk),
    .reset(reset),
    .tx_start(tx_start),
    .tx_data(tx_data),
    .tx_serial(tx_serial),
    .tx_busy(tx_busy),
    .tx_done(tx_done)
);

// UART receiver
uart_rx #(
    .CLKS_PER_BIT(CLKS_PER_BIT)
) receiver (
    .clk(clk),
    .reset(reset),
    .rx_serial(tx_serial),
    .rx_data(rx_data),
    .rx_valid(rx_valid)
);

// Task: Transmit one byte and verify the received data
task send_and_check;
    input [7:0] expected_data;
    begin
        // Wait until the transmitter is idle
        @(negedge clk);
        while (tx_busy)
            @(negedge clk);
```

```
// Apply input data and start transmission
tx_data  = expected_data;
tx_start = 1'b1;

@(negedge clk);
tx_start = 1'b0;

// Wait for receiver data with timeout protection
timeout = 0;

while ((rx_valid !== 1'b1) &&
       (timeout < 20000)) begin
   @(posedge clk);
   timeout = timeout + 1;
end

// Check result
if (rx_valid !== 1'b1) begin
   $display(
      "FAIL: Timeout waiting for data 0x%02h",
      expected_data
   );
   errors = errors + 1;
end else if (rx_data !== expected_data) begin
   $display(
      "FAIL: Expected=0x%02h, Received=0x%02h",
      expected_data,
      rx_data
```

```
            );
            errors = errors + 1;
        end else begin
            $display(
                "PASS: Transmitted=0x%02h, Received=0x%02h",
                expected_data,
                rx_data
            );
            tests_passed = tests_passed + 1;
        end
    end
endtask

// Main test sequence
initial begin
    reset       = 1'b1;
    tx_start    = 1'b0;
    tx_data     = 8'h00;
    errors      = 0;
    tests_passed = 0;
    timeout     = 0;

    // Apply reset
    repeat (5) @(negedge clk);
    reset = 1'b0;

    repeat (5) @(negedge clk);

    $display("------------------------------------");
```

```
        $display(" UART TX/RX SELF-CHECKING TEST");
        $display("-----------------------------------");

        // Test different data patterns
        send_and_check(8'h00);
        send_and_check(8'hFF);
        send_and_check(8'hA5);
        send_and_check(8'h3C);
        send_and_check(8'h81);

        // Summary
        $display("-----------------------------------");
        $display("Tests passed : %0d", tests_passed);
        $display("Errors       : %0d", errors);

        if (errors == 0) begin
            $display("FINAL RESULT: ALL TESTS PASSED");
        end else begin
            $display("FINAL RESULT: TEST FAILED");
        end

        $display("-----------------------------------");

        $finish;
    end

endmodule
```